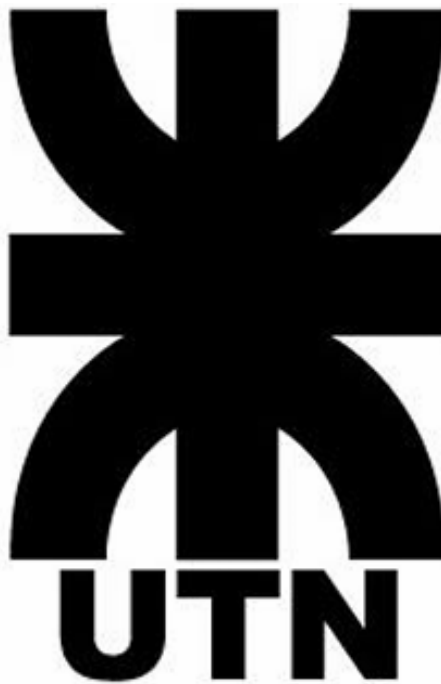


Trabajo Práctico Final

Sistemas de Información Geográfica

Simulación del monitoreo de una entidad sobre un mapa



Abonizio, Abril - 51153

Mestre, Marcos - 51898

Fecha: 24/06/2025

Índice

Índice	2
Objetivo	3
Descripción	4
Carga y visualización del mapa	5
Capas vectoriales utilizadas	6
Conversión de coordenadas geográficas	6
Agregar puntos al mapa	7
Mover punto	8
Crear ruta	9
¿Cómo se genera el recorrido óptimo?	10
Simular recorrido	11
Cargar ruta	12
Tipos y fuentes de información utilizadas	14
Conclusión	15

Objetivo

Este trabajo tiene como objetivo desarrollar una aplicación web para la simulación de monitoreo del viaje de un vehículo, permitiendo visualizar, cargar y analizar rutas a partir de datos cargados por el usuario, sea directamente en el mapa base utilizado o mediante una fuente externa, aplicando conceptos aprendidos en la materia y utilizando herramientas propias de la carrera.

Descripción

En muchas actividades productivas, logísticas y urbanas, la planificación de rutas eficientes y la visualización geográfica de datos son útiles para optimizar recursos, reducir tiempos y mejorar la toma de decisiones.

En nuestro caso, el desarrollo de esta aplicación surge de un proyecto en otra materia, el cual trata de una aplicación web de mayor magnitud en la cual las personas pueden realizar un viaje compartido y dividir los gastos del mismo sin necesidad de conocerse previamente (app web de carpooling).

Dentro de la aplicación se incluyen algunas funcionalidades, como trazar recorridos óptimos entre dos puntos o cargar y visualizar datos espaciales obtenidos de fuentes externas.

Un objetivo futuro es modificar estas funcionalidades y agregar otras, en base a las necesidades de la otra aplicación, para que las personas puedan ver en tiempo real en que parte del recorrido de su viaje se encuentran. De esta manera se busca brindar una experiencia más completa y flexible, mejorando la planificación y seguimiento de los viajes compartidos.

A continuación describimos las herramientas y tecnologías utilizadas, y las funcionalidades incluidas.

Carga y visualización del mapa

El mapa base es el núcleo de la aplicación, ya que sobre él se representan todas las funcionalidades: los puntos geográficos agregados por el usuario, las rutas generadas y la simulación de recorridos.

La aplicación utiliza una herramienta llamada *OpenLayers*, que es una librería de JavaScript especializada en mostrar mapas dentro de páginas web, permitiendo cargar capas vectoriales y ráster, mostrar mapas base, dibujar puntos, líneas y otras formas sobre él, entre otras. Cuando se abre la aplicación, se crea un "objeto mapa" (`map = new ol.Map()`) que se dibuja en un espacio de la página, y sobre él se pueden agregar diferentes elementos, como lo son las rutas, puntos, animaciones, etc.

Para mostrar el mapa base se utiliza *OpenStreetMap* (OSM), un servicio que provee imágenes de mapas reales (como calles, rutas, ríos, edificios) y que se cargan directamente desde internet. El sistema obtiene esos mapas (diferentes capas) a través de una función de OpenLayers y los posiciona en el centro que uno elija (por ejemplo, Rosario) y con un nivel de zoom determinado. En particular, la capa base cartográfica se obtiene desde OpenStreetMap (OSM) mediante el objeto `ol.source.OSM()`.

Además, como los mapas y los datos que se usan (como latitudes y longitudes) están en distintos sistemas de coordenadas, la aplicación convierte los datos del usuario al formato correcto que entiende el mapa, asegurando que todos los puntos o rutas se vean en la ubicación correcta. Estos datos son transformados al sistema de proyección EPSG:3857 mediante la función `ol.proj.fromLonLat()`.

Las funciones que nos da la librería de OpenLayers son utilizadas dentro de la lógica del manejo de la aplicación en el lenguaje JavaScript, en donde podremos agregar vectores sobre las capas, crear rutas y demás funcionalidades que veremos más en detalle.

Capas vectoriales utilizadas

Los elementos se encuentran en tres capas vectoriales separadas. Cada capa tiene una función específica:

- La capa de puntos muestra los puntos que el usuario agrega manualmente o a través de archivos.
- La capa de rutas representa la línea que une los puntos.
- La capa del vehículo utilizada para mostrar y animar el ícono que simula el recorrido sobre la ruta.

Cada elemento puede tener geometría (punto, línea, polígono) y estilo propio (color, tamaño, borde), además de que pueden modificarse (agregar, eliminar, modificar) sin afectar a las demás.

Estas capas se crean mediante herramientas de OpenLayers, como `ol.source.Vector()`, para almacenar los datos que luego se utilizan para cargarlos en una capa con `ol.layer.Vector()`.

Conversión de coordenadas geográficas

Las rutas que se cargan en la aplicación (ya sea manualmente o mediante archivos como .csv o .geojson), contienen puntos que generalmente están expresados en el sistema de coordenadas geográficas estándar WGS84 (EPSG:4326).

El mapa cargado con OpenLayers utiliza el sistema de referencia *Web Mercator* (EPSG:3857), es el sistema utilizado en aplicaciones cartográficas en línea como Google Maps o OpenStreetMap. Por esta razón fue necesario convertir todas las coordenadas para que se adapten a este sistema de proyección.

Esta transformación la realiza una función de OpenLayers llamada *ol.proj.fromLonLat()*, la cual toma como entrada una coordenada en formato [longitud, latitud] y la convierte a las coordenadas proyectadas necesarias para que los elementos se ubiquen en el lugar correcto del mapa.

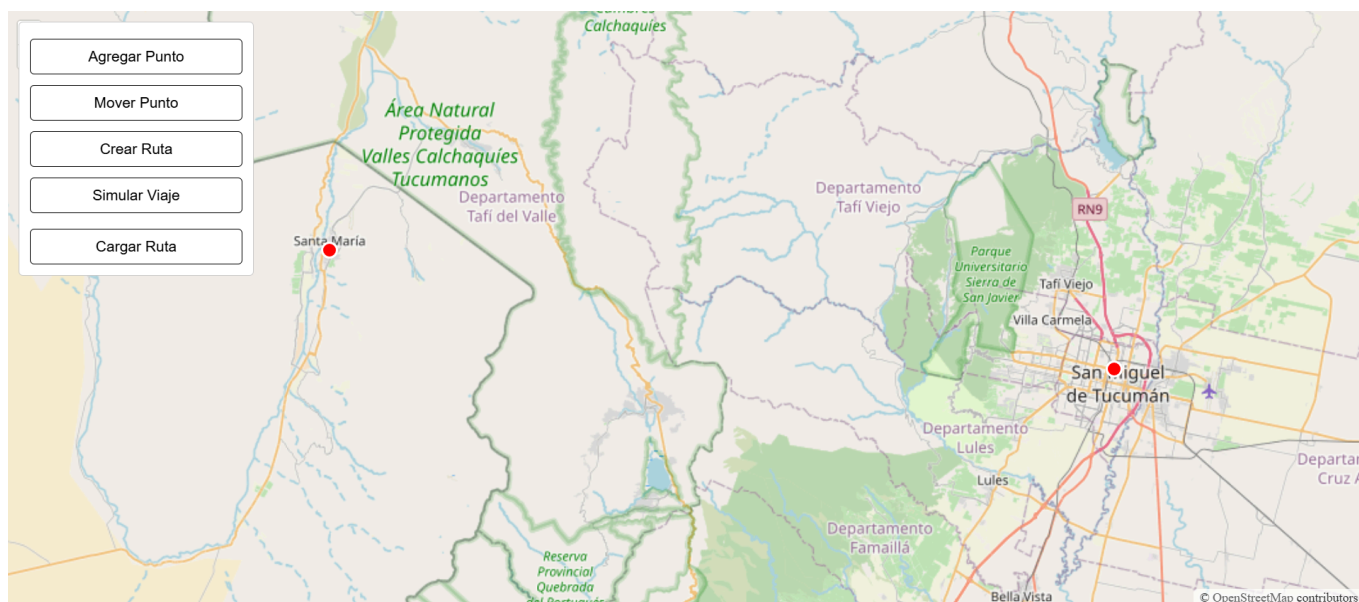
Agregar puntos al mapa

Una de las funcionalidades principales de la aplicación es permitir al usuario agregar puntos directamente sobre el mapa, permitiéndole representar ubicaciones que pueden ser utilizadas posteriormente para trazar rutas.

Cuando el usuario hace clic en el botón “Agregar Punto”, se habilita la escucha de eventos de click sobre el mapa. Cada vez que se detecta un clic, se registra la coordenada ya proyectada del lugar seleccionado y se crea un nuevo objeto geográfico tipo punto, que se añade a la capa vectorial.

Estos puntos son guardados dentro de una fuente vectorial como se mencionó anteriormente, y visualizados en el mapa con un estilo específico. La creación del elemento, la aplicación de su estilo y agregarlo a la capa son acciones que se hacen con funciones de OpenLayers manejadas dentro de la lógica en código JavaScript.

Esta funcionalidad se relaciona directamente con las demás, los puntos agregados se utilizan como inicio y destino para las rutas, y también como base para la animación del recorrido. Además, los puntos pueden ser movidos en otra funcionalidad complementaria.



Puntos generados en San Miguel de Tucuman y Santa María

Mover punto

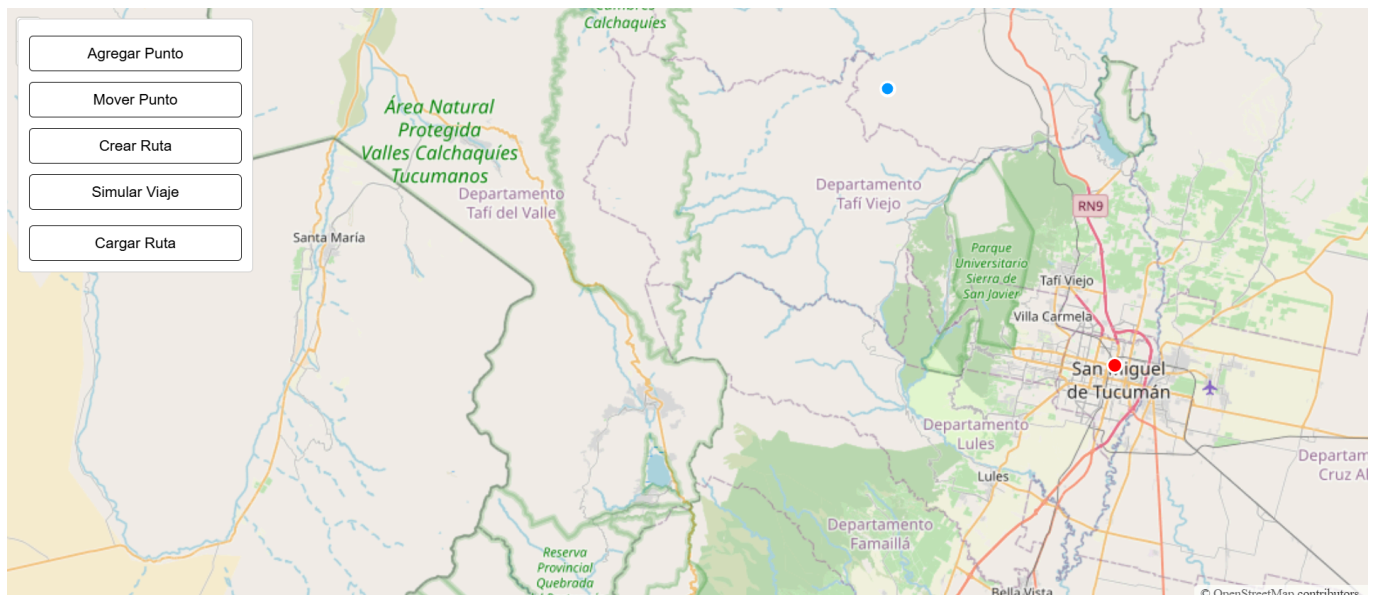
La funcionalidad de mover punto permite al usuario modificar la posición de los puntos que ya fueron agregados al mapa, algo útil para ajustar ubicaciones manualmente, sin necesidad de eliminar y volver a crear un punto.

Cuando se hace click en el botón “Mover Punto”, se desactiva la opción de agregar puntos y se activa una herramienta de modificación interactiva, proporcionada por OpenLayers, llamada Modify, la cual permite seleccionar un punto existente y arrastrarlo a otra ubicación dentro del mapa.

Internamente no se crea un nuevo punto, sino que se actualizan las coordenadas del feature existente. Como los puntos en el mapa están proyectados en el sistema Web Mercator, la nueva posición también se guarda en ese sistema de coordenadas.

Cualquier movimiento sobre el mapa se refleja inmediatamente en la fuente de datos vectorial. Además, al modificar la ubicación de los puntos, se puede volver a

generar la ruta con los nuevos valores, permitiendo recalcular trayectos fácilmente sin necesidad de volver a cargar o editar archivos.



Botón Mover Punto aplicado al punto de Santa María

Crear ruta

Esta funcionalidad permite generar un recorrido que conecta los puntos agregados en el mapa, siguiendo la lógica de un trayecto que sigue los sentidos de las calles o caminos.

Cuando se activa esta opción, la aplicación toma los puntos marcados por el usuario y extrae sus coordenadas en latitud y longitud. Esas coordenadas son enviadas a un servicio externo llamado *OSRM (Open Source Routing Machine)*, es una API pública que calcula rutas según el perfil de transporte seleccionado (vehículo, bicicleta, caminata).

El servicio devuelve una respuesta en formato *GeoJSON*, que contiene la ruta más corta como una línea. Estas coordenadas son nuevamente proyectadas al sistema EPSG:3857, para poder ser dibujadas sobre el mapa. La ruta se ve representada como una línea azul continua, trazada entre los puntos.

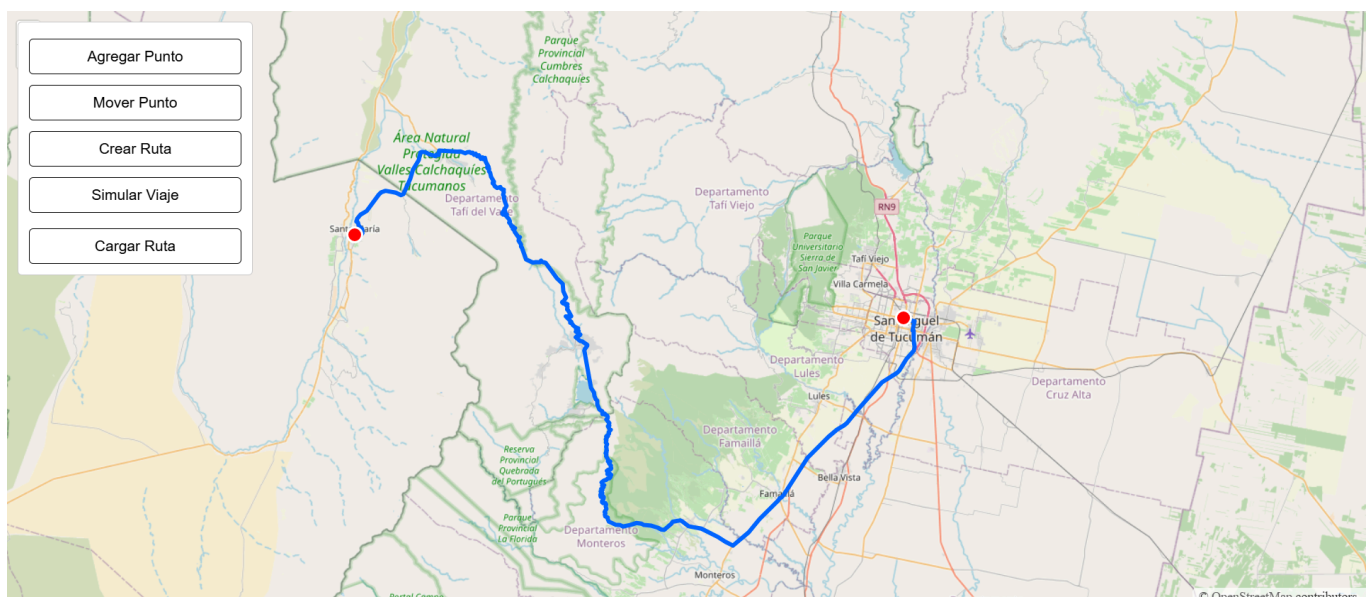
Si se quiere cambiar el tipo de trayecto (de vehículo a bicicleta o caminata), solo hay que modificar el perfil de transporte en la URL enviada al servidor OSRM (*driving, cycling, foot*).

```
`https://router.project-osrm.org/route/v1/driving/${coordinates}?overview=full&geometries=geojson`;
```

¿Cómo se genera el recorrido óptimo?

El recorrido óptimo no lo calcula la aplicación, sino que se solicita a OSRM, que utiliza datos reales de OpenStreetMap (OSM). Este servicio considera:

- La distancia total del trayecto, buscando el más corto o rápido.
- La conectividad entre calles y caminos reales.
- Restricciones de circulación, como sentidos únicos, zonas restringidas, o caminos no aptos.
- El perfil de transporte elegido (vehículo, bicicleta o caminante), que afecta el tipo de rutas disponibles.



Recorrido desde San Miguel de Tucumán hasta Santa María

Simular recorrido

Simular recorrido permite ver un desplazamiento dinámico sobre la ruta generada, representando un trayecto similar al que realiza un vehículo o persona en tiempo real. Esta simulación se realiza mediante un punto animado que avanza a lo largo del recorrido marcado en el mapa.

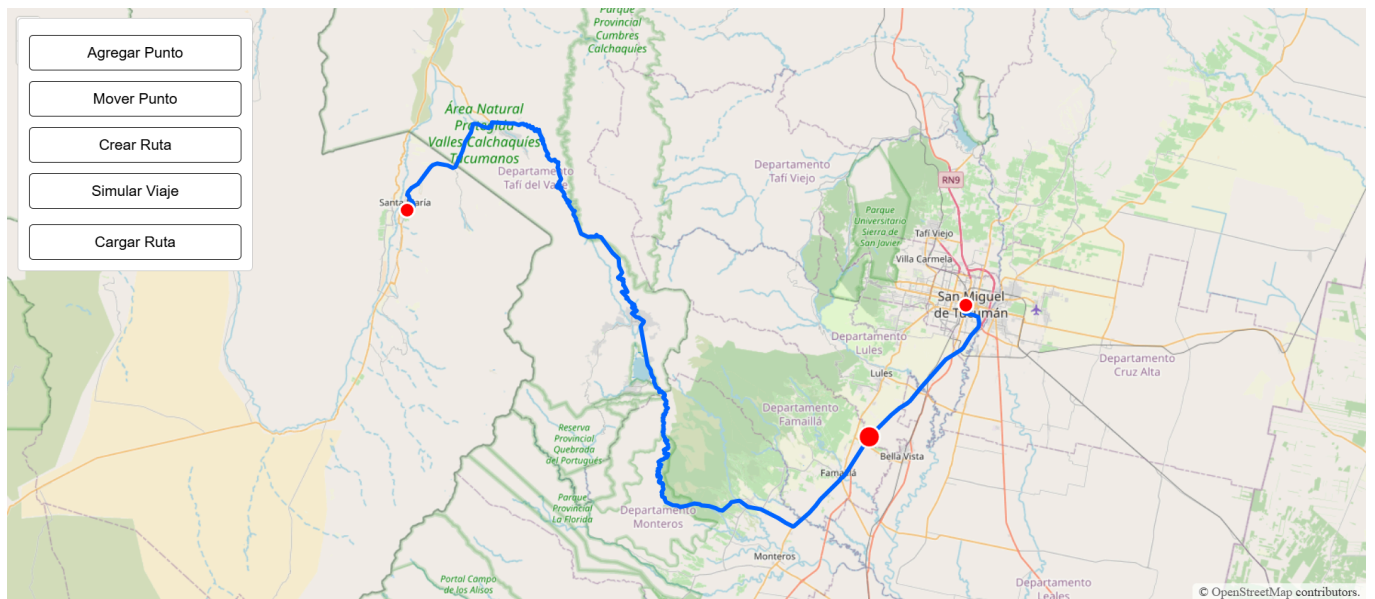
Una vez que la ruta ha sido creada, se activa esta función pulsando el botón “Simular Viaje”. Al hacerlo, la aplicación recorre paso a paso las coordenadas que forman parte del trazado de la ruta, en cada paso se actualiza la posición del ícono animado (un círculo rojo) y se redibuja en la nueva ubicación. Estas coordenadas de la ruta se encuentran dentro de un arreglo listas para usar en el mapa llamado *routeCoordinates*, para luego poder accederlas mediante un índice e ir actualizando la posición del “vehículo”.

Este movimiento se logra utilizando herramientas de JavaScript, como *requestAnimationFrame()* y *setTimeout()*, las cuales permiten controlar la velocidad del recorrido y generar una animación fluida. Además, durante la animación se calcula la rotación del ícono en función de la dirección del movimiento, en base al ángulo entre el punto actual y el siguiente.

La animación no tiene un control del tiempo real (no usa velocidades reales en km/h), sino que avanza punto por punto con una pausa entre cada uno. De todas formas, esta visualización resulta útil para comprender la trayectoria general del recorrido.

Esta funcionalidad depende de:

- La ruta previamente generada, que debe contener un conjunto ordenado de coordenadas.
- La capa del vehículo que contiene el ícono animado.
- JavaScript para manejar el avance y redibujo del punto.



Objeto animado recorriendo la ruta anteriormente creada

Cargar ruta

La funcionalidad de cargar ruta permite al usuario importar datos externos en archivos directamente en el mapa, ya sea para visualizar ubicaciones puntuales (como paradas, estaciones, parcelas) o para cargar rutas previamente definidas.

La aplicación admite tres formatos de archivos:

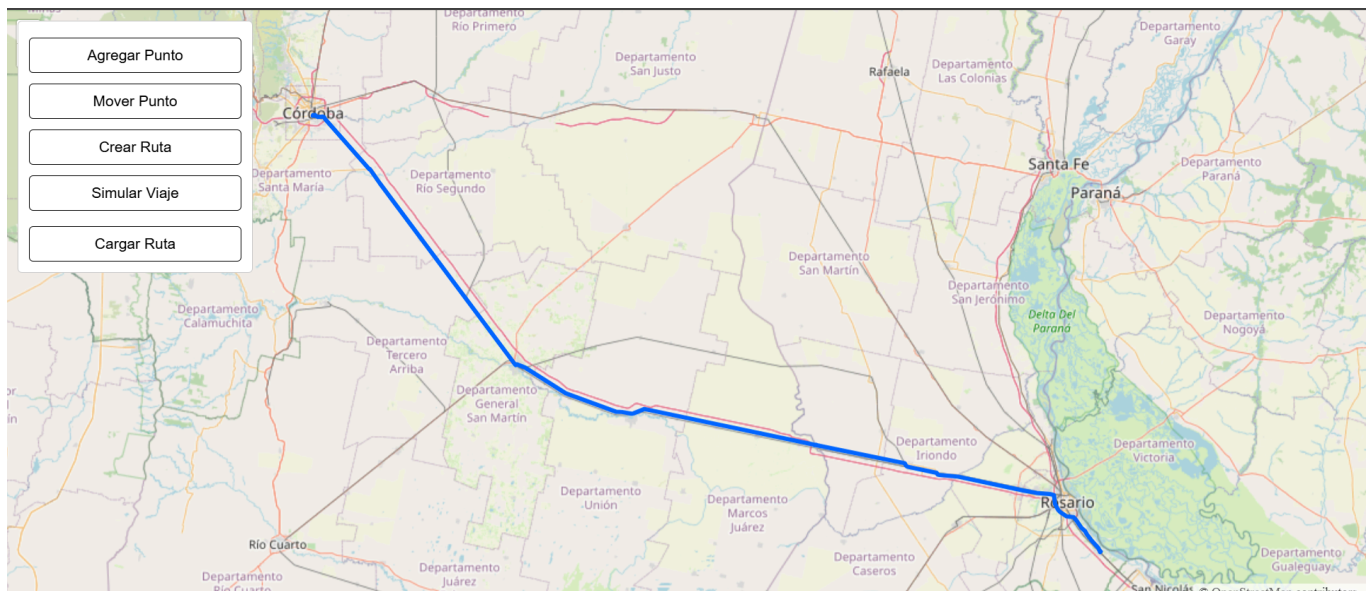
- **.csv**: archivos con coordenadas en texto plano (latitud y longitud separadas por comas).
- **.geojson y .json**: archivos en formato GeoJSON, ampliamente utilizado en sistemas geográficos, que pueden contener líneas (LineString) o puntos (Point).

Cuando el usuario selecciona un archivo, se utiliza la API FileReader de JavaScript para leer su contenido directamente en el navegador sin necesidad de conexión con

un servidor externo. Luego, se determina el tipo de archivo por su extensión y se procesa según el tipo de archivo:

- En el caso de los archivos .csv, el sistema recorre línea por línea, convierte cada par de coordenadas de latitud/longitud al sistema de proyección del mapa (`ol.proj.fromLonLat()`) y crea un punto en esa ubicación. Los puntos se agregan a la capa de puntos (`pointLayer`).
- Para archivos .geojson o .json, la aplicación busca una geometría tipo `LineString`, convierte sus coordenadas al sistema de proyección del mapa y dibuja la ruta directamente en la capa de rutas (`routeLayer`). También guarda estas coordenadas en la variable `routeCoordinates`, para que puedan ser usadas luego en la simulación.

Además, cuando se cargan datos desde un archivo, el mapa ajusta automáticamente su vista para centrar y mostrar los datos importados mediante funciones de la librería OpenLayers, permitiendo una orientación espacial inmediata.



Ruta Córdoba - Arroyo Seco cargada en formato .json

Dentro de esta funcionalidad también puede ser aplicada la funcionalidad de simular viaje.

La ruta visualizada en la imagen fue generada en la página de *ORS (Open Route Service)* y exportada en un archivo .json.

<https://maps.openrouteservice.org>

Tipos y fuentes de información utilizadas

Dentro de la aplicación se trabaja con distintos tipos de datos espaciales, incorporados manualmente o cargados desde archivos externos, los cuales representan ubicaciones, trayectos o elementos georreferenciados sobre el mapa.

- Datos puntuales (coordenadas): se cargan manualmente o se cargan desde archivos .csv.
- Datos lineales (rutas o trayectos): se generan con la API de OSRM o se cargan desde archivos .geojson.
- Datos cartográficos base: se obtienen de OpenStreetMap (OSM) y constituyen el fondo del mapa (calles, caminos, barrios, ríos).

Como mencionamos anteriormente, la aplicación soporta la carga de archivos externos en formato .csv, .geojson y .json, permitiendo así la carga de los tipos de datos antes mencionados.

Estos datos pueden ser cargados manualmente sobre el mapa, también pueden ser generados mediante otros sistemas, API's utilizadas dentro de nuestra aplicación o mediante OpenLayers.

Conclusión

El desarrollo de esta aplicación nos muestra que es posible integrar tecnologías web con herramientas SIG para realizar soluciones interactivas. A través del uso de bibliotecas como OpenLayers, servicios (OSRM), y diferentes formatos de datos (.csv y .geojson), logramos una plataforma para poder visualizar, editar, analizar y simular trayectos sobre un mapa.

Es una aplicación intuitiva, no requiere conocimientos avanzados, pero que a su vez utiliza conceptos de los Sistemas de Información Geográfica, como la gestión de capas vectoriales, la proyección de coordenadas, y la interoperabilidad de datos espaciales.

Al estar desarrollada modularmente, nos permite escalar el proyecto fácilmente, y en un futuro integrarlo con otras tecnologías sensores IoT o plataformas móviles.

Si bien la aplicación cumple con nuestro objetivo de desarrollo, existen faltantes que nos dan la oportunidad de mejorarla. La aplicación no se encuentra conectada a ningún tipo de dispositivo GPS o sensor, impidiendo que trabaje con datos en tiempo real. Todo lo que se ve reflejado sobre el mapa

Además, la falta de una conexión a una persistencia hace que la información cargada en la página se borre al recargarla, sin darle la oportunidad al usuario de tener un historial de las rutas creadas teniendo que crearlas o cargarlas nuevamente.

Estas y más limitaciones, como la falta de una velocidad realista en la simulación, el manejo de errores o la muestra de métricas, no afectan la funcionalidad, pero permitirían ampliar el alcance, la precisión y la capacidad de análisis de la aplicación, haciéndola aplicable a escenarios más diversos.